

Extension Methods in C# with Examples

In this article, I am going to discuss the **Extension Methods in C#** with Examples. Please read our previous article where we discussed [Sealed Class and Sealed Methods in C#](#) with Examples. At the end of this article, you will understand what exactly C# Extension Methods are and when and how to use these extension methods in C#.

What are Extension Methods in C#?

It is a new feature that has been added in C# 3.0 which allows us to add new methods into a class without editing the source code of the class i.e. if a class consists of a set of members in it and in the future if you want to add new methods into the class, you can add those methods without making any changes to the source code of the class.

Extension methods can be used as an approach to extending the functionality of a class in the future if the source code of the class is not available or we don't have any permission in making changes to the class.

Before extension methods, inheritance is an approach that is used for extending the functionality of a class i.e. if we want to add any new members to an existing class without making a modification to the class, we will define a child class to that existing class and then we add new members in the child class.

In the case of an extension method, we will extend the functionality of an existing class. In this case, we will create a new class and by using that new class we will extend the functionality of an existing class.

Both these approaches can be used for extending the functionalities of an existing class whereas, in the case of inheritance, we call the methods defined in the old and new classes by using the object of the new class whereas, in the case of extension methods, we call the old and new methods by using the object of the old class.

Note: Sometimes inheritance is not possible. That means if the class is declared using the sealed modifier, then we cannot create a derived class from the sealed. But we can extend the functionality of sealed class using the Extension Method.

Example to Understand Extension Methods in C#:

Let us understand Extension Methods in C# with an example. First, create a console application and then add a class file with the name OldClass.cs and then copy and paste the following code into it. As you can see in the below code, here we created the OldClass with one data member i.e. x, and two methods i.e. Test1 and Test2.

```
using System;
namespace ExtensionMethods
{
    public class OldClass
    {
        public int x = 100;
        public void Test1()
        {
            Console.WriteLine("Method one: " + this.x);
        }
    }
}
```

```

    }
    public void Test2()
    {
        Console.WriteLine("Method two: " + this.x);
    }
}

```

Now our requirement is to add three new methods to the class OldClass. For some reason, let us assume the source code is not available to us or we don't have access to the source to edit. That means we cannot change the source code of OldClass. Then how we can extend the OldClass functionality i.e. how we can add three methods to the OldClass and how we can call those new methods using the OldClass object. This is where extension methods come into the picture.

So, we can extend the functionality of OldClass with the help of extension methods. Let's create a new class with the name NewClass.cs and then copy and paste the following code into it.

```

using System;
namespace ExtensionMethods
{
    public static class NewClass
    {
        public static void Test3(this OldClass O)
        {
            Console.WriteLine("Method Three");
        }
        public static void Test4(this OldClass O, int x)
        {
            Console.WriteLine("Method Four: " + x);
        }
        public static void Test5(this OldClass O)
        {
            Console.WriteLine("Method Five:" + O.x);
        }
    }
}

```

Let us first test the application, and see whether we can access the above methods using the OldClass object or not, and then we will understand the extension methods i.e. the code written in the above class. Now to test whether the methods are accessed using the old class objects or not. Modify the Main method of the Program class as follows. Here, you can see that we are creating an instance of the OldClass and then invoking the methods which are defined inside the OldClass as well as the methods which are defined inside the NewClass, and the compiler also not giving errors.

```

using System;
namespace ExtensionMethods
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

        OldClass obj = new OldClass();
        obj.Test1();
        obj.Test2();
        //Calling Extension Methods
        obj.Test3();
        obj.Test4(10);
        obj.Test5();
        Console.ReadLine();
    }
}

```

Now, run the application and see if everything is working as expected and it will display the following output.

```

Method one: 100
Method two: 100
Method Three
Method Four: 10
Method Five:100

```

Points to Remember while working with C# Extension methods:

While working with the Extension Method in C#, we need to remember the following points.

1. Extension methods must be defined only under the static class. If you check our NewClass, then you will see that the NewClass is a static class.
2. We already discussed that Static Class in C# contains only Static Members. As an extension method is defined under a static class, it means the extension method should be created as a static method whereas once the method is bound with another class, the method changes into non-static. Now, if you check the methods in NewClass, then you will see that all three methods are declared as static only.
3. The first parameter of an extension method is known as the binding parameter which should be the name of the class to which the method has to be bound and the binding parameter should be prefixed with this As here we are creating these extension methods to extend the functionality of OldClass, so, you can check the first parameter of all these methods are going to be OldClass which is also prefixed with this keyword.
4. An extension method can have only one binding parameter and that should be defined in the first place on the parameter list.
5. If required, an extension method can be defined with normal parameters also starting from the second place of the parameter list. If you check the Test3 method, we have passed the second parameter as int and while calling this method we also need to pass one integer value.

Extension Method Real-Time Example in C#:

Let us see one real-time scenario where we can use the extension method in C#. As we know string is a built-in class provided by .NET Framework. That means the source code of this class is not available to us and hence we can change the source code of the string class. Also, the string is a sealed class in C# and hence we cannot create a derived class from the string class.

Now our requirement is to add a method to the String class i.e. GetWordCount() and that method will return the number of words present in a string and we should call this method as shown in the below image. That means by using the string object we should call that method.

```
string myWord = "Welcome to Dotnet Tutorials Extension Methods Article";
int wordCount = myWord.GetWordCount();
```

How we can achieve this? We can achieve the above requirement by using the Extension Methods in C#. First, create a class file with the name **StringExtension.cs** and then copy and paste the following code into it. As you can see, here we created the class as static and hence the GetWordCount as static and provide the first parameter as the string class name so that we can call this method on the String class object. Here, we are splitting the string by space and then we are storing each word in a string array, and then we call the LINQ Count which will return the number of elements present in the string array, and that count we are returning from the method. But if the string is empty or null then, in that case, we are returning the count as 0.

```
using System.Linq;
namespace ExtensionMethods
{
    public static class StringExtension
    {
        public static int GetWordCount(this string inputstring)
        {
            if (!string.IsNullOrEmpty(inputstring))
            {
                //Split the string by a space
                string[] strArray = inputstring.Split(' ');
                return strArray.Count();
            }
            else
            {
                return 0;
            }
        }
    }
}
```

Once you have created the extension method for the string class, then you can use that extension method on the String object. Now, let us modify the Main method of the Program class to use the string extension method to get the word count as shown in the below code.

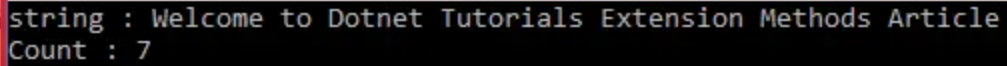
```
using System;
namespace ExtensionMethods
{
    class Program
    {
        static void Main(string[] args)
        {
            string myWord = "Welcome to Dotnet Tutorials Extension Methods Art
            int wordCount = myWord.GetWordCount();
            Console.WriteLine("string : " + myWord);
        }
    }
}
```

```

        Console.WriteLine("Count : " + wordCount);
        Console.Read();
    }
}
}

```

That's it. Now run the application and you should get the output as expected as shown in the below image.



```

string : Welcome to Dotnet Tutorials Extension Methods Article
Count : 7

```

Note: Extension methods in C# enable us to add methods to an existing type without creating a new derived type, without modifying the original type. Extension methods are static methods, but they're called as if they were instance methods on the extended type.

LINQ uses Extension Methods Effectively:

The most common extension methods are the LINQ standard query operators that add query functionality to the existing `System.Collections.IEnumerable` and `System.Collections.Generic.IEnumerable<T>` types. Once we start discussing the LINQ concept, then you will get better clarity about the Extension Methods. For a better understanding, please have a look at the following example. Here, we are using LINQ Where Method to filter the collection.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace SealedDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> intList = new List<int>()
            {
                10, 20, 30, 40, 50
            };
            //If you go to the definition of Where method, then you will see 1
            //it is implemented as an extension method
            var List1 = intList.Where(x => x > 20).ToList();
            Console.ReadKey();
        }
    }
}

```

Here, if you go to the definition of the Where method, then you will see that it is implemented as an extension method as shown in the below image.

```
public static IEnumerable<TSource> Where<TSource>(this IEnumerable<TSource> source,
Func<TSource, bool> predicate);
```

In the next article, I am going to discuss [Static Class in C#](#) with Examples. Here, in this article, I try to explain **Extension Methods in C#** with examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Extension Methods in C# with Examples article.